

UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA

ICN292 — Gestión y Rediseño de Procesos

Laboratorio 3

Automatización del triage de devoluciones de AndesHogar SpA en n8n

Estudiante	Martín Farfán Cornejo
Semilla personal S	161
Umbral de monto U	$30.000 + 1.000 \times (161 \bmod 50) = \41.000
Plazo máximo D	$7 + 7 \times (161 \bmod 4) = 14$ días
Fecha de entrega	23 de septiembre de 2026
Repositorio GitHub	https://github.com/iTechCOD/ICN292-Lab3-Farfan-Martin

Los tres valores anteriores (S, U y D) son los parámetros con los que fue construido y evaluado todo el flujo. Toda regla, condición y resultado de este informe corresponde a esa configuración.

1. Resumen ejecutivo

Se construyó en n8n un producto mínimo viable que recibe las solicitudes de devolución del canal e-commerce de AndesHogar SpA, decide automáticamente la ruta de cada una, expresa el monto en UF, notifica al cliente, deja el caso registrado y devuelve la respuesta al sistema que la envió. El trabajo se compone de tres flujos: un flujo de triage que expone un webhook y aplica la política de devoluciones, un flujo emisor que dispara las 15 solicitudes de prueba en POST hacia ese webhook, y un flujo programado que consolida el día y envía un único mensaje de resumen.

El triage aplica seis reglas en el orden exacto de la política —datos inválidos, rechazo por plazo, rechazo por estado, revisión por monto y revisión por falla— más una rama de aprobación por defecto, todo dentro de un único nodo Switch en modo Rules, sin escribir código. Con $S = 161$, el umbral quedó en $U = \$41.000$ y el plazo en $D = 14$ días. Las siete salidas del Switch confluyen en cuatro nodos Edit Fields, uno por ruta, que escriben los campos ruta y motivo; desde ahí el flujo consolida el registro auditable, consulta el valor de la UF en la API pública de mindicador.cl, notifica al cliente, registra en planilla y responde por el webhook.

Procesadas las 15 solicitudes entregadas, el resultado fue 2 aprobaciones automáticas, 9 derivaciones a revisión en bodega y 4 rechazos, sobre un monto total de \$1.251.820. La tasa de aprobación automática es de 13,3 %: con este umbral, casi todo el dinero en juego (\$1.020.890, un 81,6 % del total) sigue pasando por inspección humana. Las tres solicitudes de control obtuvieron las rutas esperadas e independientes de S , lo que confirma que la implementación aplica la política y no una interpretación propia.

La parte de errores mostró el límite real del MVP. El flujo resiste bien lo que la política sí anticipó —campos faltantes, montos no positivos, fechas fuera de plazo— y responde con una ruta y un motivo trazables. Pero se detuvo o clasificó mal en los dos casos contruidos a propósito: un monto escrito en palabras hace fallar el nodo Switch, y un SKU inexistente pasa la validación completa y se aprueba como si fuera un producto del catálogo. Ninguna regla mira el SKU y ninguna verifica que el texto recibido sea realmente un número.

Las evidencias que permiten verificar todo lo anterior son: las capturas de las ejecuciones exitosas y de la ejecución fallida con el panel de nodo, entrada y salida visible; la tabla de resultados de las 15 solicitudes con ruta y motivo; los tres flujos exportados en .json; y el repositorio GitHub público indicado en la portada, donde están todos los archivos de trabajo.

1.1 Recomendación: ¿debe este MVP convertirse en el sistema definitivo?

La recomendación es no promoverlo tal cual, pero tampoco descartarlo: conviene dejarlo operando como filtro de primera línea y construir sobre él, con tres condiciones previas.

El argumento a favor es concreto. El cuello de botella declarado era el triage —decidir si se aprueba, se revisa o se rechaza— y ese paso quedó resuelto en segundos, no en tres días hábiles. La política dejó de vivir en la cabeza de quien lee el buzón y quedó escrita en un lugar auditable, con motivo registrado para cada caso, que es exactamente lo que la regla 7 exige y lo que el proceso manual no entregaba. El compromiso de veinticuatro horas de la política publicada pasa a ser alcanzable.

El argumento en contra también es concreto. Primero, el MVP no valida los datos que recibe más allá de lo mínimo: acepta un SKU que no existe y muere frente a un monto mal escrito. Segundo, no tiene contrato con el catálogo ni con el sistema de ventas, así que no puede verificar que el monto reclamado corresponda al producto ni que la compra exista. Tercero, un solo error no controlado hace que el cliente reciba un 500 sin explicación y que el caso no quede registrado en ninguna parte, que es peor que el proceso manual: el manual es lento, pero no pierde solicitudes.

Las tres condiciones para convertirlo en definitivo son, entonces: (i) una etapa de validación de entrada antes del Switch, que rechace de forma controlada todo lo que no sea un número, una fecha o un SKU del catálogo; (ii) integración de lectura con el catálogo y el sistema de ventas, para cruzar SKU, precio y existencia de la compra; y (iii) un Error Workflow activo más un registro que se escriba antes de notificar, de

modo que ninguna solicitud desaparezca aunque falle un nodo posterior. Cumplidas esas tres, la arquitectura actual escala sin rediseño; sin ellas, el MVP sirve para descongestionar la bandeja, no para reemplazarla.

2. Parte A — Flujo de triage

2.1 Parámetros declarados

Parámetro	Cálculo	Valor
S	Tres últimos dígitos del RUT, sin dígito verificador	161
U (umbral de monto)	$30.000 + 1.000 \times (161 \bmod 50) = 30.000 + 1.000 \times 11$	\$41.000
D (plazo máximo)	$7 + 7 \times (161 \bmod 4) = 7 + 7 \times 1$	14 días

Tabla 1. Parámetros personales aplicados en las condiciones del Switch.

2.2 Arquitectura del flujo

El flujo de triage tiene once nodos. La tabla siguiente los lista en el orden de ejecución e indica qué requisito del enunciado cubre cada uno.

#	Nodo	Tipo	Función
1	Webhook Devoluciones	Webhook	Recibe la solicitud en POST sobre el path propio /devoluciones-triage-161. Modo de respuesta: "Using Respond to Webhook node".
2	Switch Triage Rules	Switch (Rules)	Aplica las seis reglas de la política en orden y deriva a una de siete salidas renombradas.
3	Set Datos Invalidos	Edit Fields	Escribe ruta = DATOS_INVALIDOS y el motivo según la causal.
4	Set Rechazo	Edit Fields	Escribe ruta = RECHAZO y el motivo (plazo o estado).
5	Set Revision	Edit Fields	Escribe ruta = REVISION y el motivo (monto o falla).
6	Set Aprobacion	Edit Fields	Escribe ruta = APROBACION y el motivo.
7	Consolidar Registro	Edit Fields	Arma el registro auditable de la regla 7: identificador, SKU, monto, ruta, motivo y los valores de U y D aplicados.
8	Consultar UF	HTTP Request	GET a la API pública de indicadores para expresar el monto en UF.
9	Notificar Cliente	Send Email	Envía al correo del cliente la ruta, el motivo y el monto en pesos y en UF.
10	Registrar Solicitud	Google Sheets	Agrega una fila al registro con todos los campos de la regla 7.
11	Responder Webhook	Respond to Webhook	Devuelve al emisor un JSON con identificador, ruta, motivo, monto, monto en UF, U y D.

Tabla 2. Nodos del flujo de triage. Cumple el mínimo de ocho nodos exigido en la Parte A.

El nodo 7 merece una aclaración, porque no aparece en el enunciado. El nodo HTTP Request reemplaza por completo el ítem que recibe con la respuesta de la API, de modo que después de consultar la UF ya no existen los campos ruta y motivo escritos en la rama. Como las cuatro ramas del Switch son mutuamente excluyentes, tampoco es posible referenciarlas después con la expresión $\$(\text{'nombre del nodo'})$: tres de los cuatro nodos no se ejecutaron y la expresión falla. Consolidar Registro resuelve el problema siendo el punto

donde las cuatro ramas convergen: es el único nodo previo a la consulta HTTP que siempre se ejecuta, y por eso los nodos 9, 10 y 11 lo usan como referencia estable.

2.3 Configuración del Switch

El Switch está en modo Rules, con cada condición en su propia regla y las salidas renombradas. Las condiciones sobre monto y días usan `.toNumber()`, porque todos los campos llegan como texto desde el formulario. El Fallback Output está configurado como rama de aprobación y la opción "Send data to all matching outputs" quedó desmarcada, de manera que la primera regla que se cumple fija la ruta y detiene la evaluación, tal como exige la regla de precedencia.

#	Condición	Detalle	Valor comparado	Salida
1	String · is empty	{{ \$json.body.id_solicitud }}	—	INVALIDO_CAMPO
2	Number · ≤	{{ \$json.body.monto.toNumber() }}	0	INVALIDO_MONTO
3	Number · >	{{ \$json.body.dias_desde_compra.toNumber() }}	14	RECHAZO_PLAZO
4	String · is equal to	estado_producto	danado_por_uso	RECHAZO_ESTADO
5	Number · >	{{ \$json.body.monto.toNumber() }}	41000	REVISION_MONTO
6	String · is equal to	estado_producto	con_fallas	REVISION_FALLA
—	Fallback Output	—	—	APROBACION

Tabla 3. Reglas del Switch en el orden de la política, con D = 14 y U = \$41.000 reemplazados por sus valores numéricos.

Las siete salidas llegan a cuatro nodos Edit Fields: las salidas 1 y 2 a datos inválidos, las salidas 3 y 4 a rechazo, las salidas 5 y 6 a revisión, y el fallback a aprobación. Se optó por mantener las condiciones separadas en lugar de unir las con el combinador OR, porque separadas permiten distinguir la causal dentro de cada rama y escribir un motivo distinto para cada una, que es lo que la regla 7 pide para que el registro sea auditable. La distinción se resuelve con una expresión ternaria dentro del Edit Fields, por ejemplo en la rama de rechazo:

```
{{ $json.body.dias_desde_compra.toNumber() > 14
  ? 'Fuera de plazo: ' + $json.body.dias_desde_compra + ' días supera D = 14'
  : 'Producto dañado por uso' }}
```

El orden importa y produce efectos observables. La solicitud DEV-1119 llega dañada por uso y además con 25 días desde la compra: como la regla de plazo se evalúa antes que la de estado, el motivo registrado es el plazo, no el daño. Lo mismo ocurre con DEV-1120 y DEV-1126, que vienen con fallas y con monto sobre el umbral: la regla de monto va primero y ese es el motivo que queda escrito. Con esta semilla la salida RECHAZO_ESTADO no recibe ninguna de las 15 solicitudes, porque los dos casos con estado dañado por uso caen antes por plazo; la rama sí existe y quedó verificada al enviar una solicitud dañada dentro del plazo.

2.4 Consulta del monto en UF

El nodo Consultar UF hace un GET sin credenciales a `https://mindicador.cl/api`. La respuesta trae todos los indicadores del día; el campo que el flujo consume es `uf.valor`, es decir el valor de la Unidad de Fomento en pesos correspondiente a la fecha de la consulta. La conversión se calcula como monto dividido por ese valor, y se escribe con cuatro decimales tanto en el registro como en la respuesta del webhook.

```
URL consumida : https://mindicador.cl/api
Método       : GET (sin autenticación)
Campo usado  : uf.valor
```

```
Expresión      : {{ $('Consolidar Registro').item.json.monto /
$json.uf.valor).toFixed(4) }}
Valor UF en la ejecución de evidencia (22-09-2026): $40.991,75
```

2.5 Notificación, registro y respuesta

La notificación al cliente se hace con un nodo Send Email dirigido al campo email_cliente de la solicitud. El mensaje informa el identificador, la ruta asignada, el motivo, el monto en pesos y su equivalente en UF, y los valores de U y D que se aplicaron. Las credenciales SMTP no están en el archivo .json: se configuran en el gestor de credenciales de n8n, como exige el enunciado.

El registro se escribe con un nodo Google Sheets en modo Append sobre la hoja "registro", con las columnas exigidas por la regla 7 más la fecha de proceso, el monto en UF y el valor de la UF utilizado. Ese mismo registro es el que lee el flujo programado de la Parte B, de modo que la trazabilidad y el resumen diario se apoyan en la misma fuente.

El flujo cierra con Respond to Webhook, que devuelve un JSON con identificador, ruta, motivo, monto, monto en UF y los parámetros aplicados. Al estar el webhook en modo "Using Respond to Webhook node", el emisor recibe esa respuesta y no un acuse genérico, lo que permite verificar la clasificación desde el propio flujo emisor.

2.6 Flujo emisor

Pegar la URL del webhook en el navegador no sirve, porque la barra de direcciones siempre hace una petición GET y el webhook está declarado en POST. Por eso las solicitudes se envían desde un segundo flujo, compuesto por un Trigger manual, un nodo Code que contiene los lotes de prueba y un HTTP Request en POST con el cuerpo en JSON. El nodo Code devuelve un ítem por solicitud y el HTTP Request se ejecuta una vez por ítem, de modo que un solo clic dispara las 15 llamadas.

El mismo nodo tiene una constante LOTE que permite cambiar entre cuatro conjuntos sin tocar el resto del flujo: base (las 15 solicitudes del curso), limites (los dos casos de frontera de la Parte C), invalidas (las tres entradas inválidas) e irresolubles (los dos casos que el flujo no puede resolver). Todas las evidencias de este informe se generaron cambiando ese único valor.

```
const LOTE = 'base'; // 'base' | 'limites' | 'invalidas' | 'irresolubles'
const lotes = { base, limites, invalidas, irresolubles };
return lotes[LOTE].map(s => ({ json: s }));
```

2.7 Las dos URL del webhook

n8n publica dos URL para el mismo nodo Webhook, con comportamientos distintos. La URL de prueba tiene la forma .../webhook-test/<path> y solo está viva mientras el flujo está abierto en el editor y se presionó "Execute workflow" o "Listen for test event": atiende una única llamada, muestra los datos entrando y saliendo de cada nodo sobre el lienzo y luego se apaga. La URL de producción tiene la forma .../webhook/<path> y solo funciona cuando el flujo está activado con el interruptor Active; atiende llamadas de forma permanente, no muestra nada en el editor y deja cada corrida en la lista de Executions.

Evidencia	URL utilizada
Lote de 15 solicitudes, casos de frontera, entradas inválidas y casos irresolubles	URL de producción (.../webhook/devoluciones-triage-161), con el flujo publicado. Las capturas se tomaron abriendo cada corrida desde la lista de Executions, donde el panel de nodo, entrada y salida queda igualmente visible.

Evidencia	URL utilizada
Experimento del error de tipo	Se intentó primero con la URL de prueba (.../webhook-test/devoluciones-triage-161), pero esa URL atiende una sola llamada y solo mientras el editor está escuchando, de modo que la ventana se cerró antes de que el emisor disparara. La evidencia definitiva se obtuvo con la URL de producción.

Esa dificultad es en sí misma la diferencia entre ambas URL: la de prueba exige coordinar manualmente el momento del disparo y sirve para inspeccionar una llamada sobre el lienzo, mientras que la de producción atiende de forma permanente y deja registro de cada corrida. Para un emisor que dispara quince llamadas seguidas, la única viable es la de producción.

Tabla 4. URL empleada en cada evidencia.

2.8 Resultados de las 15 solicitudes

La tabla siguiente muestra el resultado de procesar las 15 solicitudes con la configuración declarada. Las tres últimas filas son las solicitudes de control, cuya ruta no depende de S: en efecto, DEV-C01 se aprueba para cualquier umbral posible, DEV-C02 supera el umbral máximo alcanzable y DEV-C03 excede el plazo máximo alcanzable.

Identificador	Monto	Días	Estado	Ruta	Motivo
DEV-1115	\$19.990	3	sin_abrir	APROBACION	Dentro de plazo y bajo el umbral
DEV-1116	\$69.980	5	abierto_sin_uso	REVISION	Monto \$69.980 > U = \$41.000
DEV-1117	\$24.990	12	con_fallas	REVISION	Producto declarado con fallas
DEV-1118	\$79.990	4	sin_abrir	REVISION	Monto \$79.990 > U = \$41.000
DEV-1119	\$45.990	25	danado_por_uso	RECHAZO	25 días > D = 14 días
DEV-1120	\$89.990	2	con_fallas	REVISION	Monto \$89.990 > U = \$41.000
DEV-1121	\$289.990	9	sin_abrir	REVISION	Monto \$289.990 > U = \$41.000
DEV-1122	\$45.990	2	sin_abrir	REVISION	Monto \$45.990 > U = \$41.000
DEV-1123	\$39.980	15	danado_por_uso	RECHAZO	15 días > D = 14 días
DEV-1124	\$49.980	1	abierto_sin_uso	REVISION	Monto \$49.980 > U = \$41.000
DEV-1125	\$34.990	22	sin_abrir	RECHAZO	22 días > D = 14 días
DEV-1126	\$79.990	8	con_fallas	REVISION	Monto \$79.990 > U = \$41.000
DEV-C01	\$9.990	1	sin_abrir	APROBACION	Dentro de plazo y bajo el umbral
DEV-C02	\$289.990	2	sin_abrir	REVISION	Monto \$289.990 > U = \$41.000
DEV-C03	\$79.990	29	abierto_sin_uso	RECHAZO	29 días > D = 14 días

Tabla 5. Resultado del triage de las 15 solicitudes con S = 161, U = \$41.000 y D = 14 días.

El registro que produce el flujo, del que sale esa tabla, es el siguiente. Cada fila fue escrita por el nodo Registrar Solicitud al procesar su solicitud, con la fecha de proceso, el monto en pesos y en UF, la ruta, el motivo y los valores de U y D aplicados, tal como exige la regla 7.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
1	2026-09-22T17:00:00	DEV-1115	AU-210	19990	4.877	3 sin_abrir	APROBACION	Dentro de plazo	U_aplicado	41000	14	40991.75									
2	2026-09-22T17:00:00	DEV-1116	TE-330	69880	1.7072	5 abierto_sin_uso	REVISION	Monto \$69880 s	U_aplicado	41000	14	40991.75									
3	2026-09-22T17:00:00	DEV-1117	HW-080	24890	6.096	12 con_fallas	REVISION	Producto defectuoso	U_aplicado	41000	14	40991.75									
4	2026-09-22T17:00:00	DEV-1118	MW-150	79990	1.9514	4 sin_abrir	REVISION	Monto \$79990 s	U_aplicado	41000	14	40991.75									
5	2026-09-22T17:00:00	DEV-1119	PL-120	45990	1.1219	25 danado_por_uso	RECHAZO	Fuera de plazo	U_aplicado	41000	14	40991.75									
6	2026-09-22T17:00:00	DEV-1120	AS-260	89990	2.1953	2 con_fallas	REVISION	Monto \$89990 s	U_aplicado	41000	14	40991.75									
7	2026-09-22T17:00:00	DEV-1121	LV-330	269990	7.0744	9 sin_abrir	REVISION	Monto \$269990 s	U_aplicado	41000	14	40991.75									
8	2026-09-22T17:00:00	DEV-1122	PL-120	45990	1.1219	2 sin_abrir	REVISION	Monto \$45990 s	U_aplicado	41000	14	40991.75									
9	2026-09-22T17:00:00	DEV-1123	AU-210	39880	9.753	15 danado_por_uso	RECHAZO	Fuera de plazo	U_aplicado	41000	14	40991.75									
10	2026-09-22T17:00:00	DEV-1124	HW-080	49880	1.2193	1 abierto_sin_uso	REVISION	Monto \$49880 s	U_aplicado	41000	14	40991.75									
11	2026-09-22T17:00:00	DEV-1125	TE-330	34990	8.536	22 sin_abrir	RECHAZO	Fuera de plazo	U_aplicado	41000	14	40991.75									
12	2026-09-22T17:00:00	DEV-1126	MW-150	79990	1.9514	8 con_fallas	REVISION	Monto \$79990 s	U_aplicado	41000	14	40991.75									
13	2026-09-22T17:00:00	DEV-C01	CB-010	9990	2.437	1 sin_abrir	APROBACION	Dentro de plazo	U_aplicado	41000	14	40991.75									
14	2026-09-22T17:00:00	DEV-C02	LV-330	269990	7.0744	2 sin_abrir	REVISION	Monto \$269990 s	U_aplicado	41000	14	40991.75									
15	2026-09-22T17:00:00	DEV-C03	MW-150	79990	1.9514	29 abierto_sin_uso	RECHAZO	Fuera de plazo	U_aplicado	41000	14	40991.75									

Figura 4b. Hoja de registro al terminar el lote: las 15 solicitudes con su ruta, su motivo y el monto convertido a UF con el valor de \$40.991,75 vigente ese día.

2.9 Consolidado del lote

Ruta	Solicitudes	Monto total	Participación del monto
APROBACION	2	\$29.980	2,4 %
REVISION	9	\$1.020.890	81,6 %
RECHAZO	4	\$200.950	16,1 %
Total	15	\$1.251.820	100,0 %

Tabla 6. Consolidado por ruta. Tasa de aprobación automática: 13,3 %.

El dato relevante para la gestión es que solo 13,3 % de las solicitudes se resuelve sin intervención humana, y que el 81,6 % del monto sigue derivándose a revisión en bodega. El umbral $U = \$41.000$ es bajo respecto de los precios del catálogo —seis de los ocho SKU cuestan más que eso—, de modo que la regla de monto domina la clasificación: ocho de las nueve revisiones se explican por monto y solo una por falla declarada. Si la empresa quisiera aumentar la resolución automática, el parámetro a discutir es el umbral, no el flujo.

2.10 Capturas de ejecución

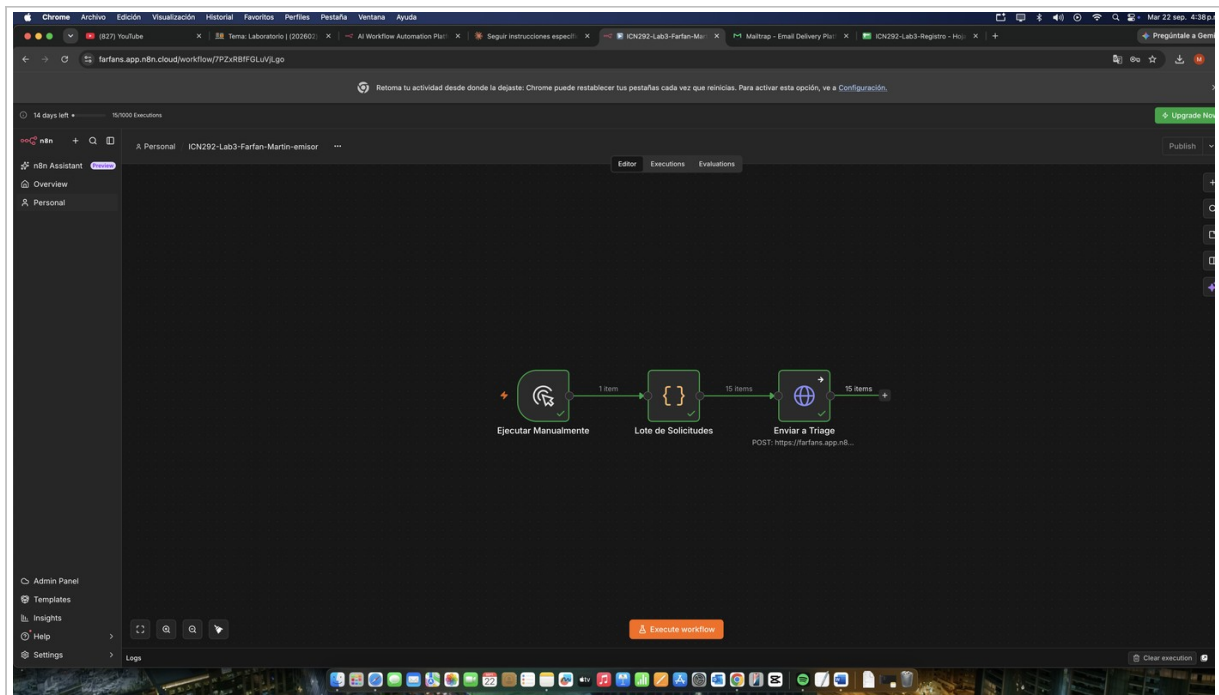


Figura 1. Flujo emisor: el nodo Lote de Solicitudes entrega 15 ítems y el HTTP Request los envía uno a uno a la URL de producción del webhook.

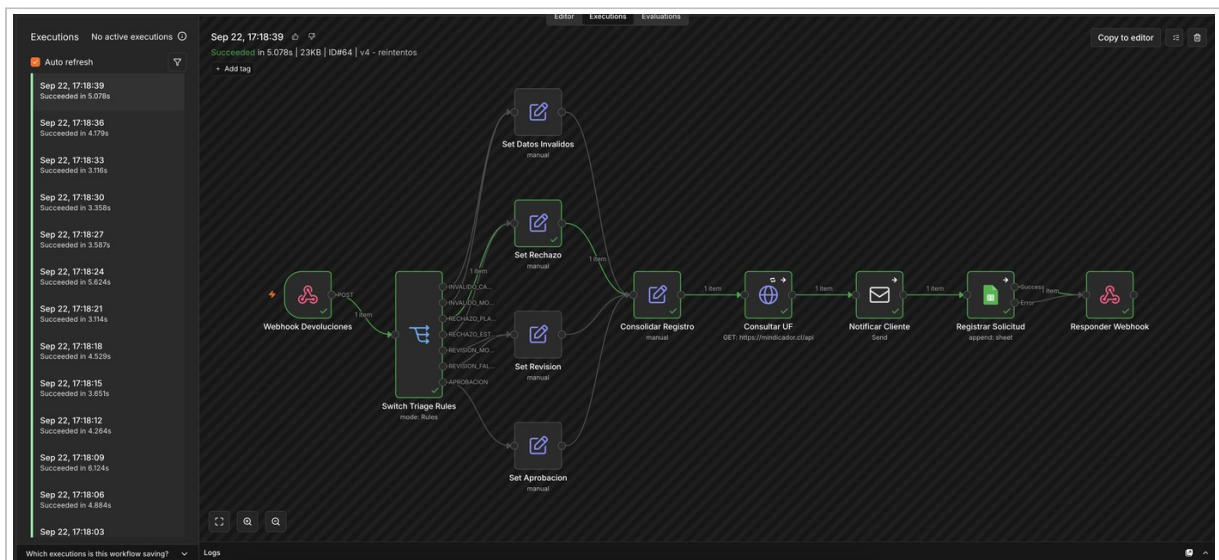


Figura 2. Ejecución exitosa del triage (ID#64, 5,078 s). A la izquierda, la lista de Executions con las quince corridas del lote. En el lienzo, el recorrido completo: Webhook, Switch, Set Rechazo, Consolidar Registro, Consultar UF, Notificar Cliente, Registrar Solicitud y Responder Webhook.

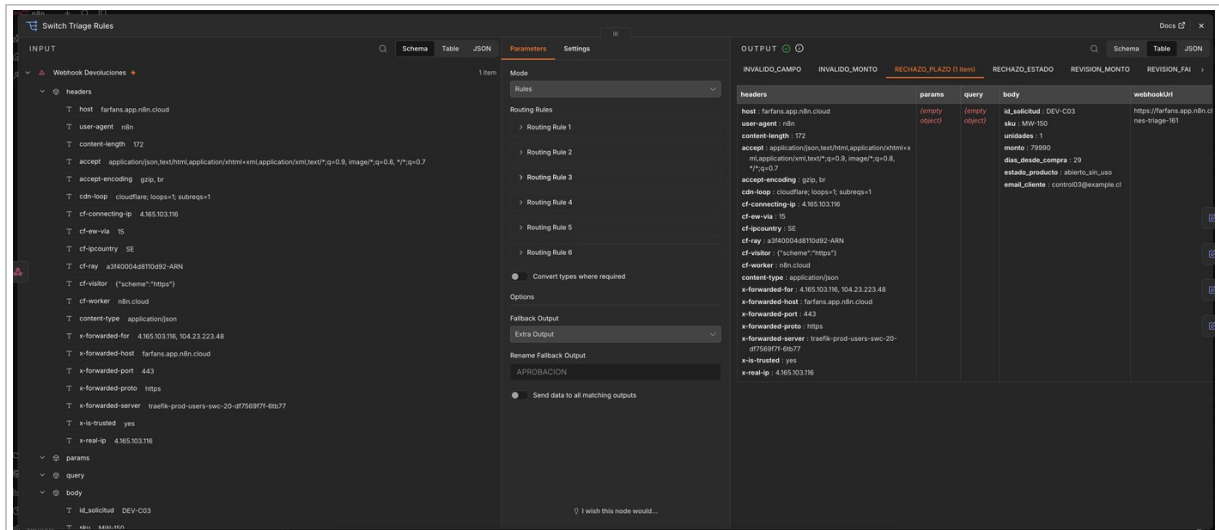


Figura 3. Detalle del nodo Switch Triage Rules con entrada y salida visibles. El INPUT muestra el body de DEV-C03; la configuración, las seis reglas, el Fallback Output renombrado a APROBACION y "Send data to all matching outputs" desmarcado; el OUTPUT, las siete salidas con RECHAZO_PLAZO activa.

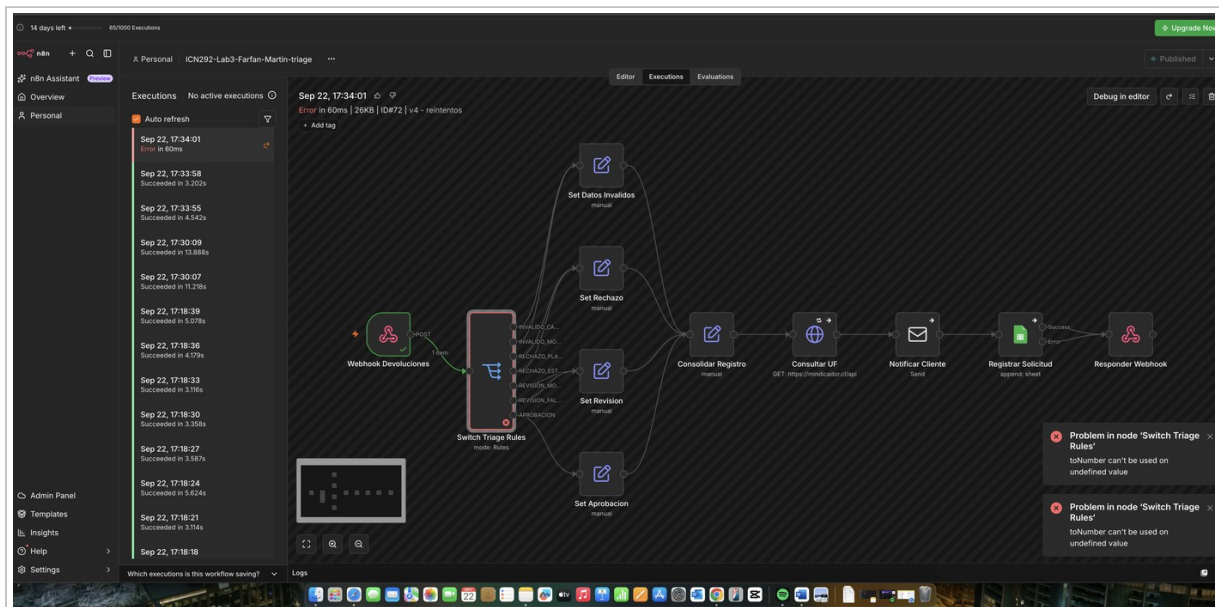


Figura 4. Ejecución fallida (ID#72, error en 60 ms) provocada por la entrada inválida E3. El nodo Switch Triage Rules queda marcado en rojo y el flujo no alcanza a registrar ni a notificar.

3. Parte B — Flujo programado de resumen

El tercer flujo se ejecuta solo, lee el registro que dejó el triage, consolida el día y envía un único mensaje. Sus nodos son los siguientes.

#	Nodo	Tipo	Función
1	Schedule Diario 19:00	Schedule Trigger	Dispara el flujo todos los días a las 19:00, hora de Santiago.
2	Leer Registro	Google Sheets	Lee todas las filas de la hoja registro.
3	Filtrar Día de Hoy	Filter	Deja solo las filas cuya fecha de proceso corresponde al día en curso.

#	Nodo	Tipo	Función
4	Hubo Solicitudes	IF	Separa el caso con movimiento del caso sin movimiento.
5	Summarize por Ruta	Summarize	Agrupar por ruta y calcula cantidad de solicitudes y suma de montos.
6	Consolidar Día	Aggregate	Reúne las filas del Summarize en un solo ítem para poder enviar un mensaje único.
7	Componer Resumen	Edit Fields	Arma el asunto y el cuerpo del mensaje con los cuatro indicadores pedidos.
8	Componer Aviso Sin Movimiento	Edit Fields	Arma el mensaje alternativo para un día sin solicitudes.
9	Enviar Resumen	Send Email	Envía un solo correo a la jefatura de Servicio al Cliente.

Tabla 7. Nodos del flujo programado de resumen.

3.1 Contenido del mensaje

El mensaje reporta las cuatro cosas exigidas: solicitudes por ruta, monto total por ruta, monto total del día y tasa de aprobación automática. Aplicado al lote de 15 solicitudes, el mensaje que produce el flujo es el siguiente.

```

RESUMEN DIARIO DE DEVOLUCIONES - AndesHogar SpA
Fecha: 22-09-2026 | S = 161 | U = $41.000 | D = 14 días

SOLICITUDES Y MONTO POR RUTA
APROBACION: 2 solicitudes | $29.980
REVISION: 9 solicitudes | $1.020.890
RECHAZO: 4 solicitudes | $200.950

Monto total del día: $1.251.820
Solicitudes totales: 15
Tasa de aprobación automática: 13,3 %

```

3.2 Frecuencia elegida

Se eligió una ejecución diaria a las 19:00. La razón es que el destinatario del mensaje es la jefatura de Servicio al Cliente y el uso que le va a dar es de gestión, no de operación: revisar cuánta carga hubo, cuánto dinero quedó detenido en revisión y si la tasa de aprobación se movió. Esa decisión no se toma varias veces al día. La hora responde a que el turno de atención cierra antes, de modo que a las 19:00 el día ya está completo y el resumen no queda a medias; ejecutarlo a medianoche habría complicado la lectura de "hoy" y ejecutarlo al mediodía habría obligado a enviar dos mensajes.

Una frecuencia más alta —cada hora, por ejemplo— no aportaría información nueva: el volumen es de quince solicitudes en el lote de prueba y un correo por hora se transformaría en ruido que nadie abre. Una frecuencia más baja —semanal— llegaría tarde para detectar un día anómalo, que es justamente lo que este mensaje permite ver.

3.3 Qué ocurre un día sin solicitudes

Sin ninguna precaución, un día sin solicitudes produce silencio, y ese es el problema: el filtro no deja pasar ningún ítem, los nodos siguientes no se ejecutan, no se envía correo y la ejecución queda marcada como

exitosa. Para quien espera el resumen, un día sin movimiento y un flujo caído se ven exactamente igual, porque en ambos casos no llega nada.

Por eso el flujo lo trata de forma explícita. El nodo Filter tiene activada la opción "Always Output Data", de modo que cuando no hay coincidencias igual emite un ítem vacío; el nodo IF verifica si ese ítem trae un identificador de solicitud y, si no lo trae, deriva a la rama Componer Aviso Sin Movimiento. Esa rama envía el mismo correo con el texto "No se recibieron solicitudes de devolución en el día. El flujo se ejecutó correctamente". Así, la ausencia de solicitudes se comunica como un hecho, y la ausencia de correo vuelve a significar lo único que debería significar: que algo falló.

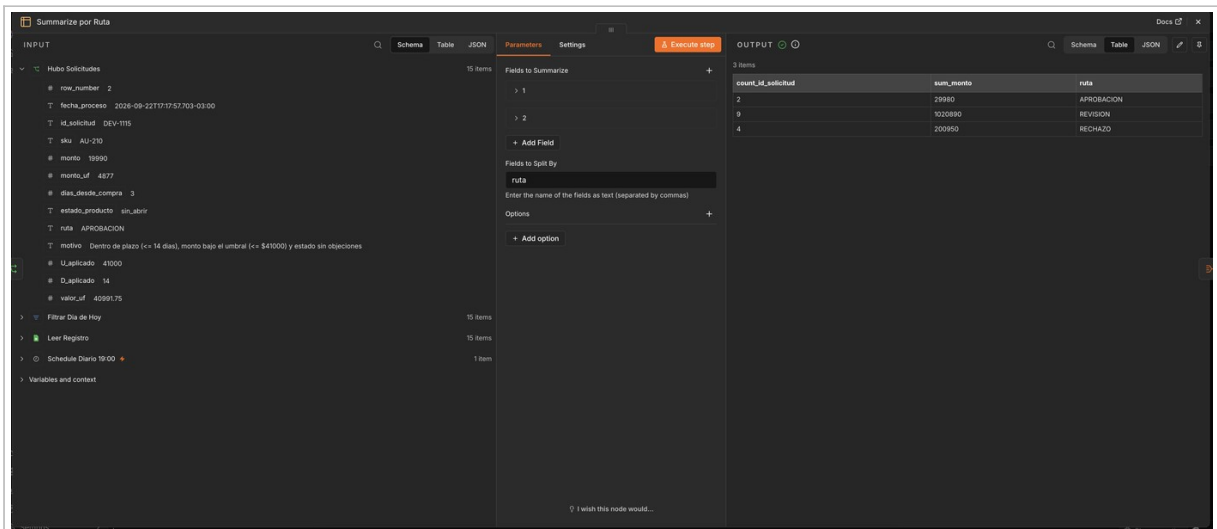


Figura 5. Nodo Summarize por Ruta. Entran los 15 registros del día y salen tres filas agrupadas por ruta, con la cantidad de solicitudes y la suma de montos de cada una.

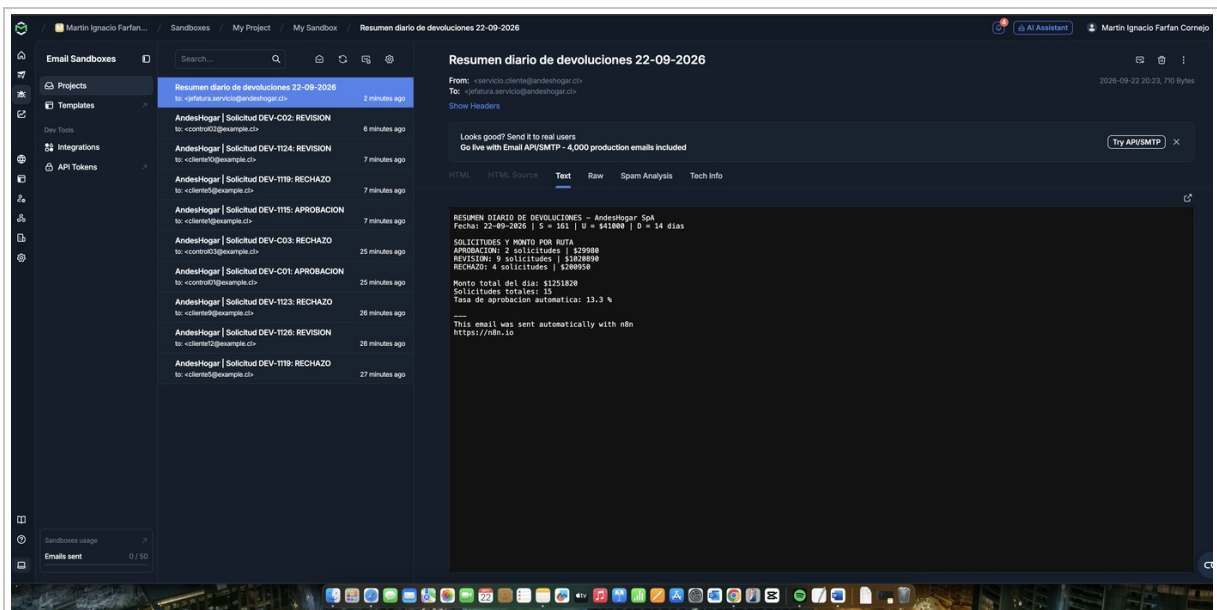


Figura 6. Mensaje único recibido en la bandeja de pruebas, con los cuatro indicadores exigidos. Sobre él se ven las notificaciones individuales enviadas a cada cliente por el flujo de triage.

4. Parte C — Errores, bordes y límites contruidos

4.1 Tres entradas inválidas

Se enviaron tres cargas inválidas distintas desde el flujo emisor, con el lote invalidas. Las dos primeras están previstas por la política y toman la ruta de datos inválidos; la tercera no está prevista y hace fallar un nodo.

#	Carga enviada	Comportamiento esperado	Comportamiento observado
E1	<pre>{ "id_solicitud": "", "sku": "HV-050", "monto": "24990", "dias_desde_compra": "3", "estado_producto": "sin_abrir" }</pre>	La regla 1 detecta el identificador vacío y deriva a INVALIDO_CAMPO. El flujo responde con ruta DATOS_INVALIDOS.	Ruta DATOS_INVALIDOS, motivo "Falta el campo obligatorio id_solicitud". Respuesta HTTP 200 con el JSON de la ruta. Ningún nodo falla.
E2	<pre>{ "id_solicitud": "DEV- E02", "sku": "TE-330", "monto": "0", "dias_desde_compra": "2", "estado_producto": "abierto_sin_uso" }</pre>	La regla 2 detecta que el monto no es mayor que cero y deriva a INVALIDO_MONTO.	Ruta DATOS_INVALIDOS, motivo "El monto informado (0) no es mayor que cero". Respuesta HTTP 200. Ningún nodo falla.
E3	<pre>{ "id_solicitud": "DEV- E03", "sku": "MW-150", "dias_desde_compra": "5", "estado_producto": "sin_abrir" } (falta por completo el campo monto)</pre>	La política no contempla un campo ausente: la expresión de la regla 2 intenta convertir un valor inexistente y el nodo debería detenerse.	El nodo Switch Triage Rules falla al evaluar la regla 2, porque .toNumber() se aplica sobre un valor undefined. La ejecución queda marcada como fallida, el webhook devuelve error y la solicitud no se registra ni se notifica.

Tabla 8. Entradas inválidas enviadas, comportamiento esperado y observado. El caso E3 es el que hace fallar un nodo.

Figura 7. Caso E1 en el nodo Set Datos Invalidos. El OUTPUT muestra id_solicitud vacío y el nodo escribe ruta DATOS_INVALIDOS con el motivo "Falta el campo obligatorio id_solicitud". La expresión ternaria que distingue la causal dentro de la rama queda visible en el panel central.

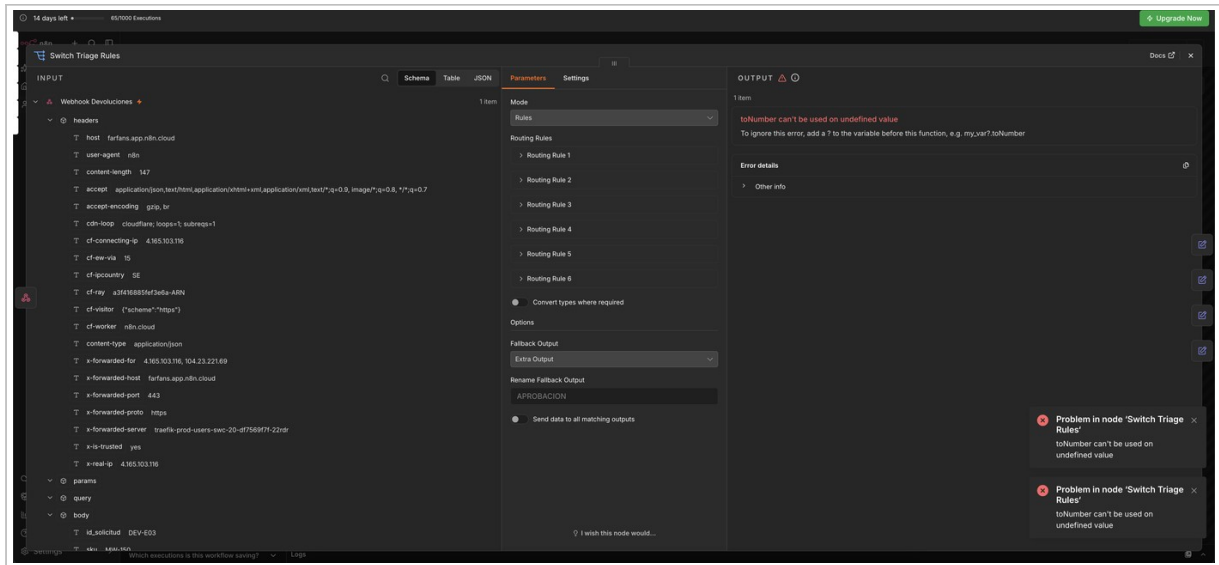


Figura 8. Caso E3 en el nodo Switch Triage Rules. El mensaje es "toNumber can't be used on undefined value": la regla 2 intenta convertir un campo monto que no existe en la carga y el nodo se detiene.

E3 deja a la vista una diferencia que importa para el diseño: la ruta de datos inválidos no es lo mismo que un error. La ruta es una decisión de negocio, queda registrada y el cliente recibe una explicación; el error es una caída técnica que no deja rastro útil, no notifica a nadie y devuelve un código de error al emisor. Un dato malo que sabemos nombrar se convierte en ruta; un dato malo que no anticipamos se convierte en caída.

4.2 Los tres valores de On Error

Cada nodo de n8n tiene, en su pestaña Settings, un parámetro On Error con tres valores posibles:

- Stop Workflow: el nodo detiene la ejecución completa. Es el valor por defecto y el más estricto.
- Continue (using error output): el nodo suma una segunda salida por la que se van los ítems que fallaron, mientras los correctos siguen por la salida normal. Permite tratar el fallo con nodos propios.
- Continue (using regular output): el ítem que falló sigue por la salida normal con un campo error incorporado. El flujo no se detiene y el fallo queda registrado dentro del dato.

La asignación por nodo en el flujo de triage fue la siguiente.

Nodo	On Error elegido	Razón
Webhook Devoluciones	Stop Workflow	Si la petición no se puede recibir no hay nada que procesar; continuar solo generaría un registro vacío.
Switch Triage Rules	Stop Workflow	Es el nodo que decide. Si no logra clasificar, dejarlo continuar significaría que la solicitud caiga al fallback y se apruebe por accidente, que es el peor desenlace posible.
Los cuatro Edit Fields	Stop Workflow	Escriben ruta y motivo. Un registro sin motivo no es auditable según la regla 7, así que es preferible detener y corregir.
Consolidar Registro	Stop Workflow	Misma razón: es el nodo que arma el registro exigido por la regla 7.

Nodo	On Error elegido	Razón
Consultar UF	Continue (using regular output), con 3 reintentos	El monto en UF es información complementaria. Si mindicador.cl no responde, la clasificación sigue siendo válida y no tiene sentido botar la solicitud por un indicador externo. Los reintentos cubren la caída momentánea.
Notificar Cliente	Continue (using regular output)	Un correo que no sale es molesto, pero perder el registro de la solicitud es mucho peor. El flujo continúa y la fila igual se escribe.
Registrar Solicitud	Continue (using error output)	El registro sí es crítico, pero la falla debe poder tratarse: la salida de error se conecta igualmente a la respuesta del webhook, de modo que el emisor recibe la clasificación y el fallo de escritura queda visible en la ejecución en vez de detener todo en silencio.
Responder Webhook	Stop Workflow	Es el último nodo; si falla, no hay nada posterior que rescatar.

Tabla 9. Valor de On Error elegido en cada nodo del flujo de triage.

El criterio detrás de toda la tabla es uno solo: se detiene el flujo cuando el nodo produce la decisión o el registro, y se deja continuar cuando el nodo produce un complemento. La clasificación y la trazabilidad no admiten degradación; el indicador en UF y el correo sí.

4.3 Qué es un Error Workflow

Un Error Workflow es un flujo aparte que arranca con un nodo Error Trigger y que se designa, en la pestaña Settings de otro flujo, como el flujo que debe ejecutarse cuando ese otro falla. n8n lo llama automáticamente y le entrega los datos de la ejecución caída: el nombre del flujo, el nodo que falló, el mensaje de error y el enlace a la ejecución. Ahí uno decide qué hacer con esa información: enviarla por correo, dejarla en un canal de mensajería o abrir un ticket.

Un flujo programado sin Error Workflow deja de funcionar en silencio porque nadie está esperando su respuesta. Un webhook falla y el que llamó recibe un código de error de inmediato: hay alguien del otro lado que se entera. Un Schedule Trigger, en cambio, no tiene interlocutor: si el nodo que lee el registro falla porque cambió el nombre de una hoja o expiró una credencial, la ejecución queda marcada como fallida en una lista que nadie mira, no se envía ningún correo y el destinatario simplemente no recibe el resumen. Y como el flujo ya contempla los días sin solicitudes, la ausencia de mensaje no llama la atención: se parece a un día tranquilo. El Error Workflow es lo que convierte esa falla muda en un aviso.

4.4 Los límites: monto igual a U y días iguales a D

Las 15 solicitudes entregadas no incluyen ningún caso de frontera: ninguna tiene monto exactamente igual a \$41.000 ni exactamente 14 días desde la compra. Se construyeron dos solicitudes adicionales para verificar la regla 6, que declara las comparaciones inclusivas: monto igual a U se aprueba y días iguales a D está en plazo.

Identificador	Monto	Días	Estado	Ruta obtenida	Verificación
DEV-L01	\$41.000	3	sin_abrir	APROBACION	La regla 5 compara monto > \$41.000 en sentido estricto, así que \$41.000 no la cumple y la solicitud cae al Fallback Output. El monto exactamente igual al umbral queda aprobado.

Identificador	Monto	Días	Estado	Ruta obtenida	Verificación
DEV-L02	\$19.990	14	sin_abrir	APROBACION	La regla 3 compara días > 14 en sentido estricto, así que 14 días no la cumple. El plazo exacto queda dentro de lo aceptado.

Tabla 10. Casos de frontera contruidos. Ambos quedan dentro de lo aceptado, como exige la regla 6.

La inclusividad no se logró con una condición aparte, sino eligiendo el operador correcto: usar "mayor que" en lugar de "mayor o igual que" en las reglas 3 y 5 es lo que deja el borde adentro. Si esas dos reglas se hubieran escrito con $\geq$, DEV-L01 habría ido a revisión y DEV-L02 habría sido rechazada, y la política publicada se habría incumplido en el caso exacto en que el cliente reclama.

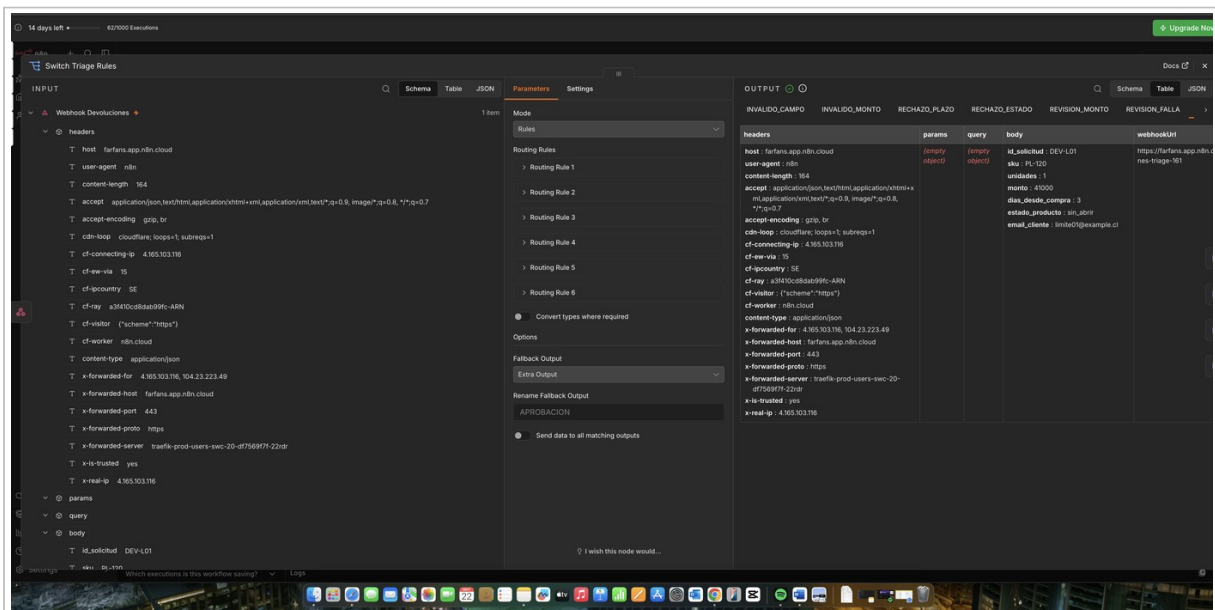


Figura 9. Caso DEV-L01 con monto exactamente igual a U. El body muestra monto 41000 y la solicitud sale por el Fallback Output, es decir APROBACION: la regla 5 compara en sentido estricto y el umbral exacto queda aprobado.

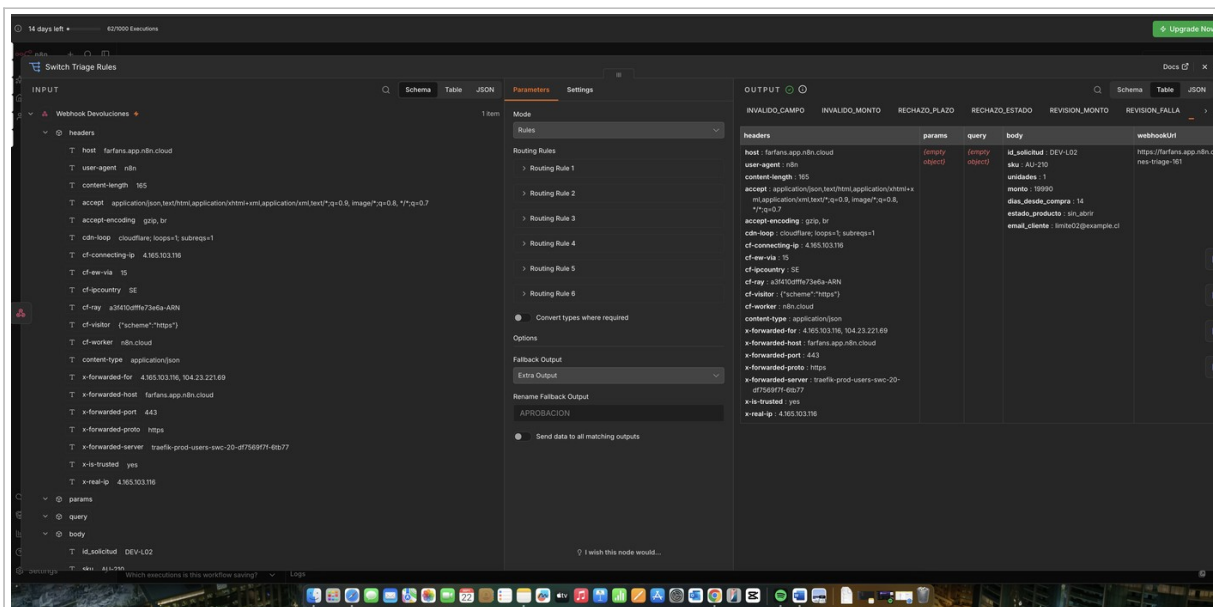


Figura 10. Caso DEV-L02 con plazo exactamente igual a D. El body muestra dias_desde_compra 14 y la solicitud sale también por el Fallback Output: la regla 3 compara en sentido estricto y el plazo exacto queda dentro de lo aceptado.

4.5 El error de tipo, a propósito

Se cambió el tipo de la condición del monto en la regla 5, de Number a String, y se envió la solicitud de control DEV-C01, cuyo monto es \$9.990. El experimento dejó dos hallazgos, uno antes de ejecutar y otro al ejecutar.

El primero apareció al abrir el desplegable del operador. El tipo String de n8n no ofrece ninguna comparación de orden: sus operaciones son exists, does not exist, is empty, is not empty, is equal to, is not equal to, contains, does not contain, starts with, ends with y las de expresión regular. No existe un "mayor que" para texto. Es decir, la regla de la política —monto mayor que el umbral— es literalmente inexpresable con tipo String, y para forzar el cambio hubo que elegir la operación más cercana, is equal to contra el valor 41000.

El segundo apareció al enviar DEV-C01. El Switch no llegó a comparar nada: se detuvo con un error de validación de tipo y el mensaje fue explícito.

```
Problem in node 'Switch Triage Rules'
Wrong type: '9990' is a number but was expecting a string [condition 0, item 0]

Ejecucion ID#80 | Error in 66 ms | ruta obtenida: ninguna
```

La ruta que obtiene DEV-C01 es, entonces, ninguna. La ejecución muere en 66 milisegundos, contra los 3 a 5 segundos que tardan las corridas exitosas: el flujo no alcanza a registrar ni a notificar, y el emisor recibe un error en lugar de una clasificación. Una solicitud perfectamente válida de \$9.990 desaparece del sistema.

La explicación en dos líneas es esta. El tipo de la condición, no el del dato, decide cómo se compara: el valor llega como número por efecto de .toNumber(), la condición declarada espera texto, y n8n valida el tipo antes de evaluar la comparación, de modo que detiene el nodo en vez de arriesgar un resultado sin sentido. Y si esa validación no existiera y la comparación se resolviera como texto, el orden sería alfabético y no numérico: "9990" resultaría mayor que "41000" porque el carácter 9 va después del 4, con lo que una devolución de nueve mil pesos terminaría derivada a revisión en bodega.

Verificado el efecto, la condición se revirtió a Number con .toNumber(), que es la configuración del archivo entregado.

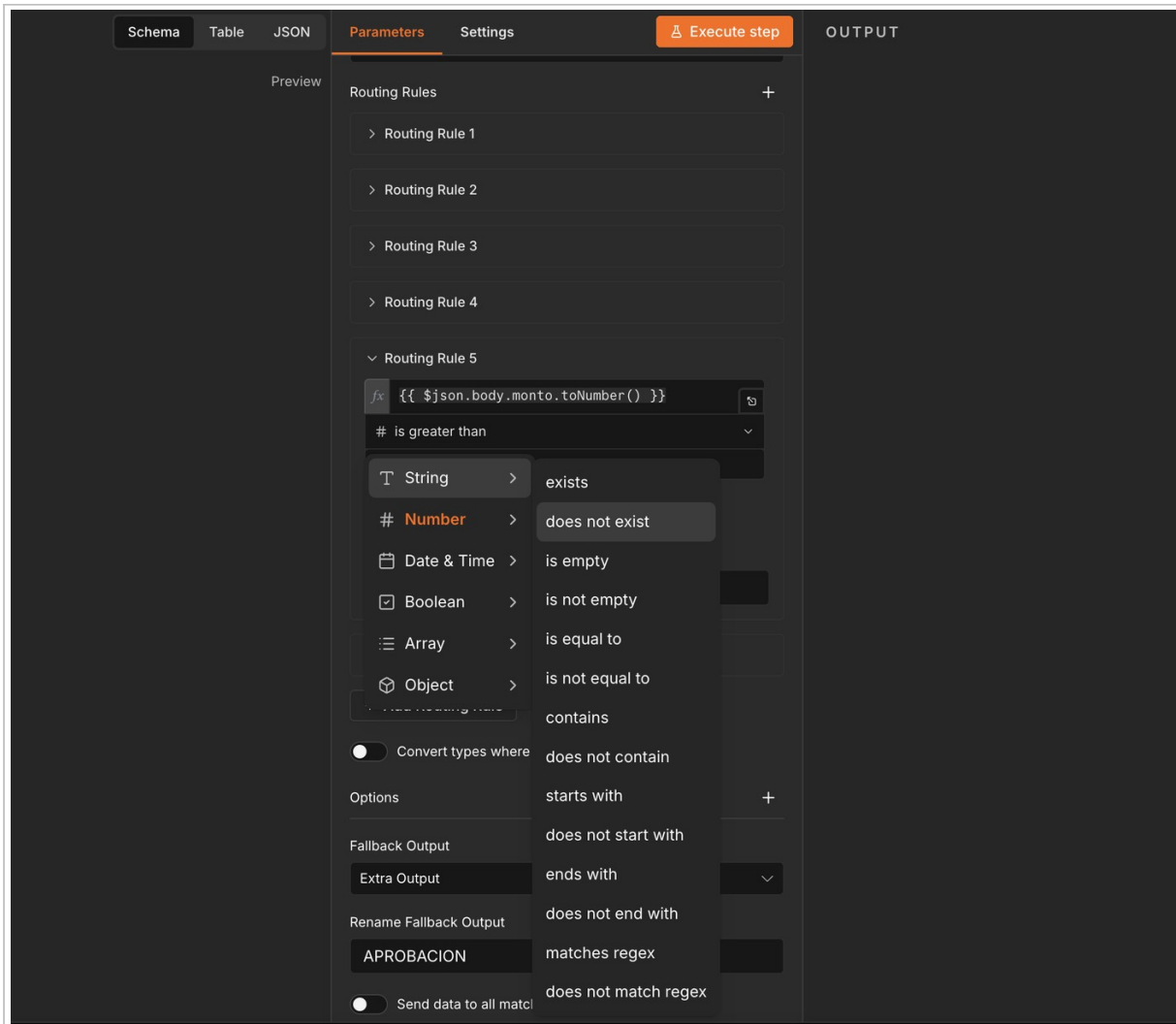


Figura 13. Operadores que n8n ofrece para el tipo String. La lista no incluye ninguna comparación de orden, de modo que la regla "monto mayor que el umbral" no puede expresarse con ese tipo.

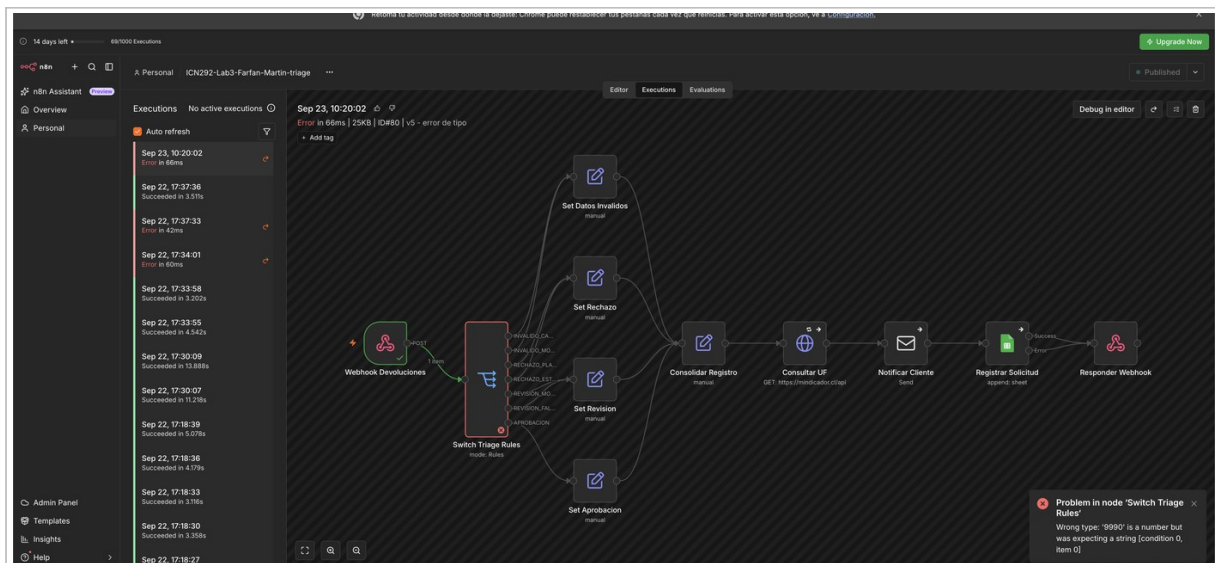


Figura 14. Ejecución ID#80 con la condición del monto declarada como String. El Switch se detiene en 66 ms con el mensaje "Wrong type: 9990 is a number but was expecting a string" y DEV-C01 no obtiene ninguna ruta.

4.6 Los dos casos que el flujo no puede resolver

Caso 1 — "monto": "veinte mil".

El nodo Switch falla al evaluar la regla 2, porque `.toNumber()` aplicado a un texto que no es numérico no produce un número válido y la validación estricta del nodo rechaza la comparación. Ocurre porque el flujo valida la forma de los datos —que el campo exista, que el número sea positivo— pero nunca valida que el texto recibido sea efectivamente un número, y el formulario web entrega todo como texto sin comprobarlo. Para detectarlo, el flujo necesitaría una regla previa a todas las demás que pregunte si la conversión del monto es un número válido y, si no lo es, derive a `INVALIDO_MONTO` como decisión de negocio en vez de caer como falla técnica.

Caso 2 — "sku": "ZZ-999".

La solicitud pasa la validación completa y se clasifica como si fuera válida: con monto de 29.990 pesos, 4 días desde la compra y estado sin abrir, sale por el Fallback Output y queda aprobada. Ocurre porque ninguna de las seis reglas mira el SKU: la política se escribió sobre monto, plazo y estado, y el flujo no tiene forma de saber que ese código no existe en el catálogo. Para detectarlo, el flujo necesitaría consultar el catálogo antes del Switch —una lectura contra la hoja catalogo o contra el sistema de ventas— y una regla que derive a datos inválidos cuando el SKU no aparece, idealmente comprobando además que el monto informado sea coherente con el precio del producto por la cantidad de unidades.

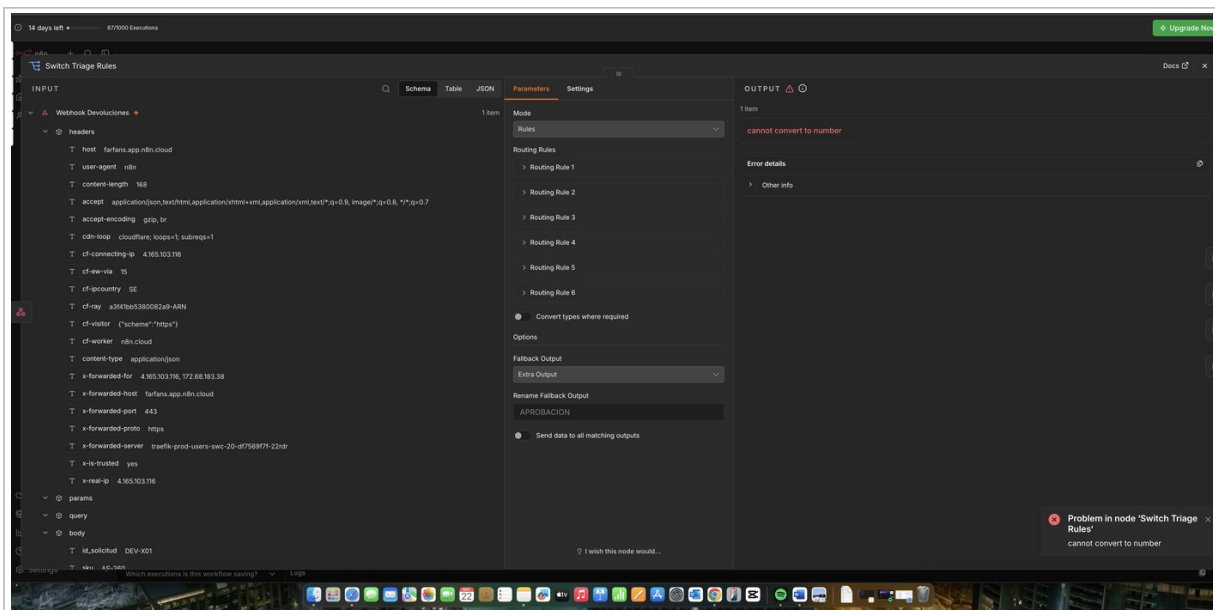


Figura 11. Caso DEV-X01 con monto "veinte mil". El Switch se detiene con el mensaje "cannot convert to number": la solicitud no obtiene ruta, no se registra y no se notifica.

The screenshot displays the Switch Triage Rules interface. On the left, the 'INPUT' section lists various rules and their configurations. The central panel shows the 'Rules' configuration, including 'Routing Rules' and 'Fallback Output'. The 'OUTPUT' section on the right shows the results of the rules, including headers, parameters, query, body, and webhook data. The 'Fallback Output' is set to 'APROBACION'.

headers	params	query	body	webhookUrl
host: farfars.app.nhn.cloud			id_solicitud: DEV-X02	https://farfars.app.nhn.cloud/nes-triage-161
user-agent: nbn			sku: ZZ-999	
content-length: 163			unidades: 1	
accept: application/json, text/html, application/xhtml+xml, application/xml, text/*; q=0.8, image/*; q=0.7			monto: 29990	
accept-encoding: gzip, br			dias_desde_compra: 4	
cdn-loop: cloudflare, loops=1, subreqs=1			estado_producto: sin_abrir	
cf-connecting-ip: 4.165.103.116			email_cliente: border02@example.cl	
cf-ew-via: 15				
cf-ipcountry: SE				
cf-ray: a348bc78a760d92-ABN				
cf-visitor: {"scheme":"https"}				
cf-worker: nbn.cloud				
content-type: application/json				
x-forwarded-for: 4.165.103.116, 104.23.223.49				
x-forwarded-host: farfars.app.nhn.cloud				
x-forwarded-port: 443				
x-forwarded-proto: https				
x-forwarded-server: trafik-prod-users-svc-20-df7569f7f-b6b77				
x-is-trusted: yes				
x-real-ip: 4.165.103.116				

Figura 12. Caso DEV-X02 con SKU ZZ-999. La solicitud pasa la validación completa (monto 29.990, 4 días, sin abrir) y sale por el Fallback Output como APROBACION, pese a que el código de producto no existe en el catálogo.

5. Bonus — La versión con código

Se implementó además una variante del flujo de triage en la que un único nodo Code reemplaza al Switch y a los cuatro Edit Fields, con exactamente las mismas seis reglas y el mismo orden de precedencia. El resto del flujo —consulta de UF, notificación, registro y respuesta— se mantiene sin cambios. Esta variante no reemplaza la Parte A: el flujo principal entregado sigue siendo el que usa Switch Rules.

```
if (id === undefined || String(id).trim() === '') { ruta = 'DATOS_INVALIDOS'; ... }
else if (!(monto > 0)) { ruta = 'DATOS_INVALIDOS'; ... }
else if (dias > D) { ruta = 'RECHAZO'; ... } // D = 14
else if (estado === 'danado_por_uso') { ruta = 'RECHAZO'; ... }
else if (monto > U) { ruta = 'REVISION'; ... } // U = 41000
else if (estado === 'con_fallas') { ruta = 'REVISION'; ... }
else { ruta = 'APROBACION'; ... }
```

5.1 Comparación de ambas implementaciones

Cuál es más fácil de auditar. El Switch, sin discusión. Un jefe de Servicio al Cliente puede abrir el nodo, ver las seis reglas ordenadas en pantalla con sus salidas renombradas y comprobar que corresponden a la política sin saber programar. La versión en Code exige leer una cadena de condicionales y confiar en que el orden de los else if refleja la precedencia; el orden sigue estando ahí, pero deja de ser visible y pasa a ser una convención del lenguaje. Además, el Switch deja huella gráfica de por dónde salió cada ítem, mientras que el Code entrega el resultado ya cocinado y hay que confiar en él.

Cuál es más fácil de mantener. Depende de qué cambie. Para modificar un umbral, agregar una regla o reordenar dos causales, el Switch se edita en el navegador en un minuto y no requiere pruebas de sintaxis. Para cambios que involucran lógica compuesta —una regla que dependa de tres campos a la vez, un cálculo intermedio, una consulta al catálogo antes de decidir— el Code es mucho más corto: lo que en Switch obliga a agregar ramas, nodos y conexiones, en Code son tres líneas. El Switch también se degrada con el tamaño: siete salidas ya ocupan buena parte del lienzo, y quince serían inmanejables.

Cuál elegiría si la política cambiara cada mes. Elegiría el Switch. El supuesto de un cambio mensual es que quien cambia la regla no es quien construyó el flujo: es alguien del área de negocio ajustando un umbral o el orden de dos causales. Ese cambio, en el Switch, es abrir el nodo y escribir un número, con la política a la vista para verificar que quedó bien; en Code es editar JavaScript en un cuadro de texto, donde una coma perdida rompe el triage completo y el error recién aparece en ejecución. La ventaja del Code aparece cuando la lógica se vuelve compleja, no cuando cambia seguido. Si además la política cambiara de forma compleja cada mes, la respuesta correcta ya no sería ninguna de las dos, sino sacar los parámetros del flujo y dejarlos en una tabla que el negocio edite sin tocar n8n.

6. Archivos entregados

Todos los archivos están publicados en el repositorio público indicado en la portada, y son los mismos que se suben a Aula USM.

Archivo	Contenido
ICN292-Lab3-Farfan-Martin.pdf	Este informe en PDF.
ICN292-Lab3-Farfan-Martin.docx	El mismo informe en Word.
ICN292-Lab3-Farfan-Martin-triage.json	Flujo de triage exportado desde n8n (Parte A).
ICN292-Lab3-Farfan-Martin-emisor.json	Flujo emisor que envía las solicitudes en POST.
ICN292-Lab3-Farfan-Martin-resumen.json	Flujo programado de resumen diario (Parte B).
ICN292-Lab3-Farfan-Martin-triage-code.json	Variante del triage con un único nodo Code (Bonus).
README.md	Nombre, semilla S, fecha e instrucciones para importar y ejecutar cada archivo.
capturas/	Capturas de las ejecuciones exitosas y de la ejecución fallida.

Tabla 11. Archivos de la entrega. Ninguno de los .json contiene contraseñas, tokens ni URL con credenciales, y todos tienen pinData vacío.